

Vector Vanguard

Arcade Survival Shooter in C + SDL2

Pankaj

Software Engineering Demo

June 10, 2026

`github.com/Pankaj809/vector-vanguard`

What Is Vector Vanguard?

In plain English

You pilot a ship in a grid arena. Red enemies chase you. Shoot them before they hit you. Survive and score points.

Genre

- Single-player arcade shooter
- Top-down survival
- 3 difficulty levels

Built with

- C99 + SDL2
- 1024×768 display
- Sound + music + HUD log

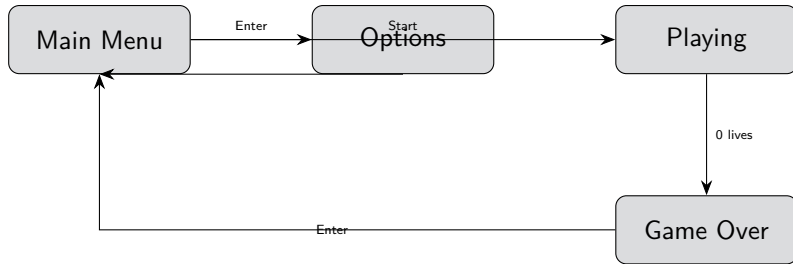
How to Play

Action	Keys
Move ship	WASD or Arrow keys
Fire laser	Space or Left Ctrl
Menu navigate	Up / Down
Confirm	Enter
Go back	Esc

Goal

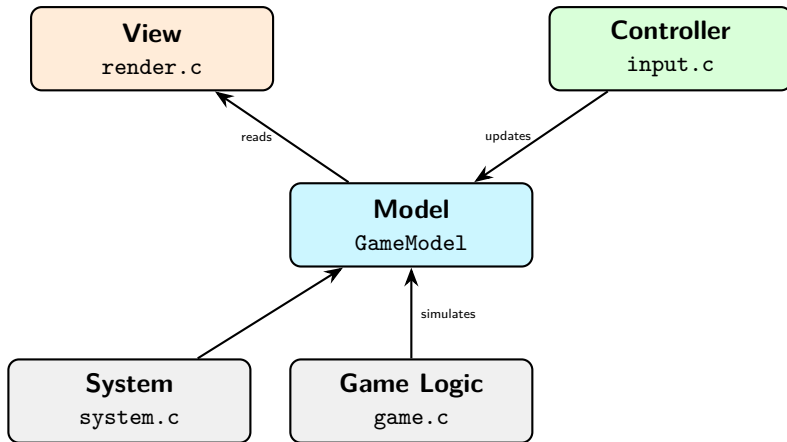
Destroy enemies (+10 points each). You have **3 lives**. Beat your **high score**. Don't let hostiles reach your ship.

Game Flow



Difficulty (Options): Easy = slower enemies; Hard = faster spawns.

Architecture: MVC Pattern



One struct (`GameModel`) holds **all game state**. Modules never own duplicate data.

The Model: GameModel

Stores everything:

- Screen state (menu, playing, game over)
- Score, lives, difficulty
- Player, enemies, bullets (entities)
- Particle effects
- Combat log messages
- Audio + font handles

Why one model?

Easy to reason about.

Easy to test.

No hidden global variables.

Memory pool

Up to 512 entities pre-allocated — no malloc during gameplay.

View vs Controller

View — render.c

- Draws grid + HUD
- Renders ship, enemies, bullets
- Shows menus & combat log
- **Never** changes game rules

Controller — input.c

- Reads keyboard
- Menu selection
- Moves player velocity
- Fires bullets
- **Never** draws pixels

main.c runs the loop: **Input** → **Update** → **Render** at 60 FPS

Game Logic (Simple Terms)

- 1 **Spawn** — Every few seconds a new enemy appears at the top.
- 2 **Chase** — Each enemy moves toward your ship.
- 3 **Shoot** — Your bullets fly in the direction you last moved.
- 4 **Hit test** — Circles overlap? Enemy dies (+10) or you lose a life.
- 5 **Respawn** — Still have lives? Ship returns to center.
- 6 **Game Over** — No lives left; high score saved to disk.

Fair physics

Game updates in fixed **1/60 second** steps so speed is identical on every computer.

Engineering Highlights

- **Separation of concerns** — MVC modules in separate `.c` files
- **Deterministic loop** — Accumulator pattern in `main.c`
- **Asset pipeline** — Fonts & audio loaded via `system.c`
- **Portable build** — CMake + `run.sh` for one-command start
- **Standalone deploy** — `package.sh` bundles binary + assets

Vector Vanguard

Arcade fun + clean C architecture

Clone & play:

```
git clone https://github.com/Pankaj809/vector-vanguard.git  
./run.sh
```

Questions?